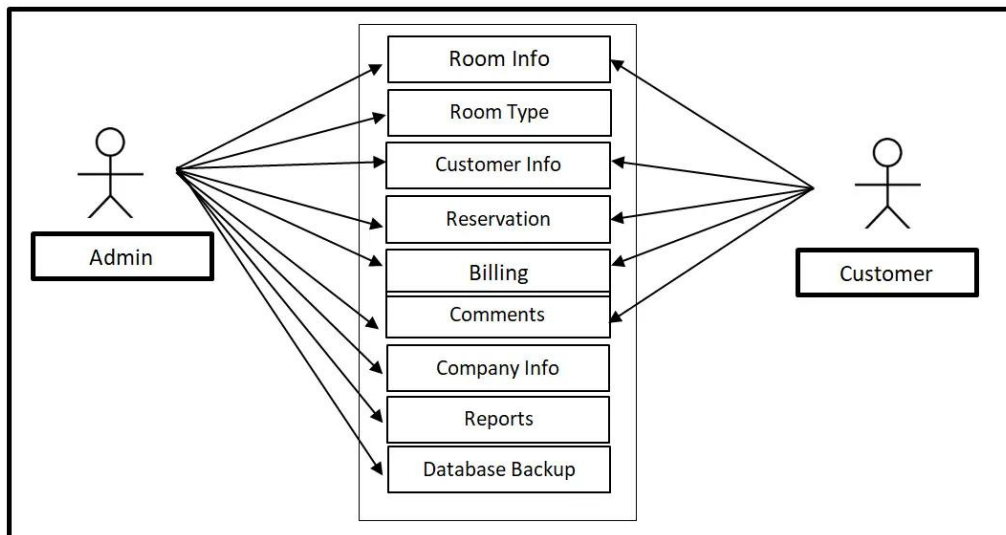


STE PTT 2 Question Bank

Unit 3

1. Sketch Use-Case diagram for Hotel Management System with minimum two actors and four use cases.

Ans.



2. Differentiate between Black Box Testing and White Box Testing. (Six points)

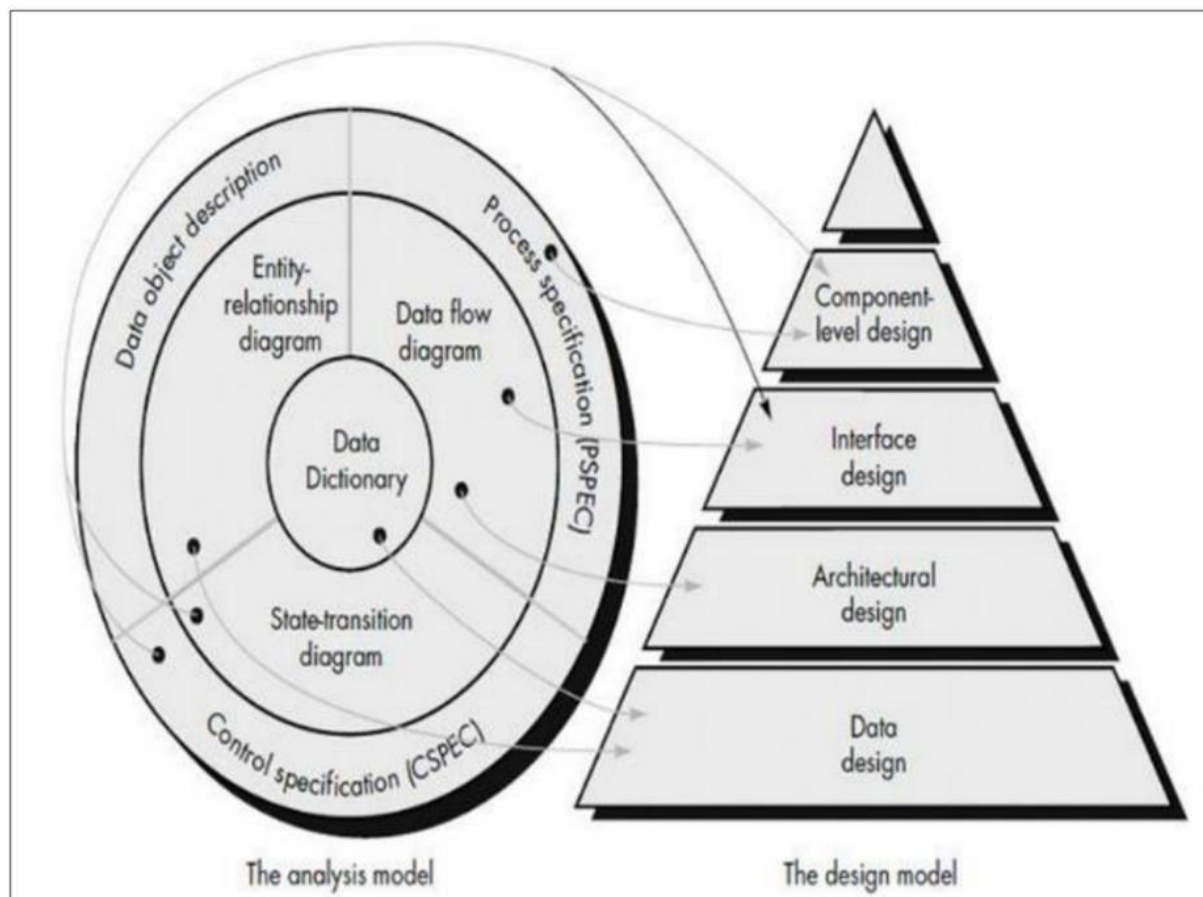
Ans.

Parameters	Black Box Testing	White Box Testing
Definition	Black Box Testing is a way of software testing in which the internal structure or the program or the code is hidden and nothing is known about it.	White Box Testing is a way of testing the software in which the tester has knowledge about the internal structure or the code or the program of the software.
Testing objectives	Black box testing is mainly focused on testing the functionality of the software, ensuring that it meets the requirements and specifications.	White box testing is mainly focused on ensuring that the internal code of the software is correct and efficient.
Scope	Black box testing is generally used for testing the software at the functional level.	White box testing is used for testing the software at the unit level, integration level and system level.
Implementation	Implementation of code is not needed for black box testing.	Code implementation is necessary for white box testing.
Done By	Black Box Testing is mostly done by software testers.	White Box Testing is mostly done by software developers.

Terminology	Black Box Testing can be referred to as outer or external software testing.	White Box Testing is the inner or the internal software testing.
Testing Level	Black Box Testing is a functional test of the software.	White Box Testing is a structural test of the software.
Testing Initiation	Black Box testing can be initiated based on the requirement specifications document.	White Box testing of software is started after a detail design document.
Programming	No knowledge of programming is required.	It is mandatory to have knowledge of programming.
Testing Focus	Black Box Testing is the behavior testing of the software.	White Box Testing is the logic testing of the software.

3. Draw neat labelled diagram for translation of requirement model into design model.

Ans.



4. Explain decision table with example. Accessing secured network.

Ans.

Decision Table – Accessing Secured Network

A **Decision Table** is a tabular method used to represent **conditions and corresponding actions** for different situations. It is useful when a system has multiple conditions and decisions.

Example: Accessing a Secured Network

Conditions:

- C1: Username is valid
- C2: Password is valid
- C3: OTP is valid

Actions:

- A1: Allow network access
- A2: Deny network access

Conditions / Rules	R1	R2	R3	R4
Valid Username	Y	Y	N	Y
Valid Password	Y	N	Y	Y
Valid OTP	Y	Y	Y	N
Actions				
Allow Access	✓	x	x	x
Deny Access	x	✓	✓	✓

Explanation

- **R1:** All three credentials are valid → **Access is allowed.**
- **R2:** Password is invalid → **Access is denied.**
- **R3:** Username is invalid → **Access is denied.**
- **R4:** OTP is invalid → **Access is denied.**

Thus, a decision table clearly shows **all possible combinations of conditions and the resulting actions.**

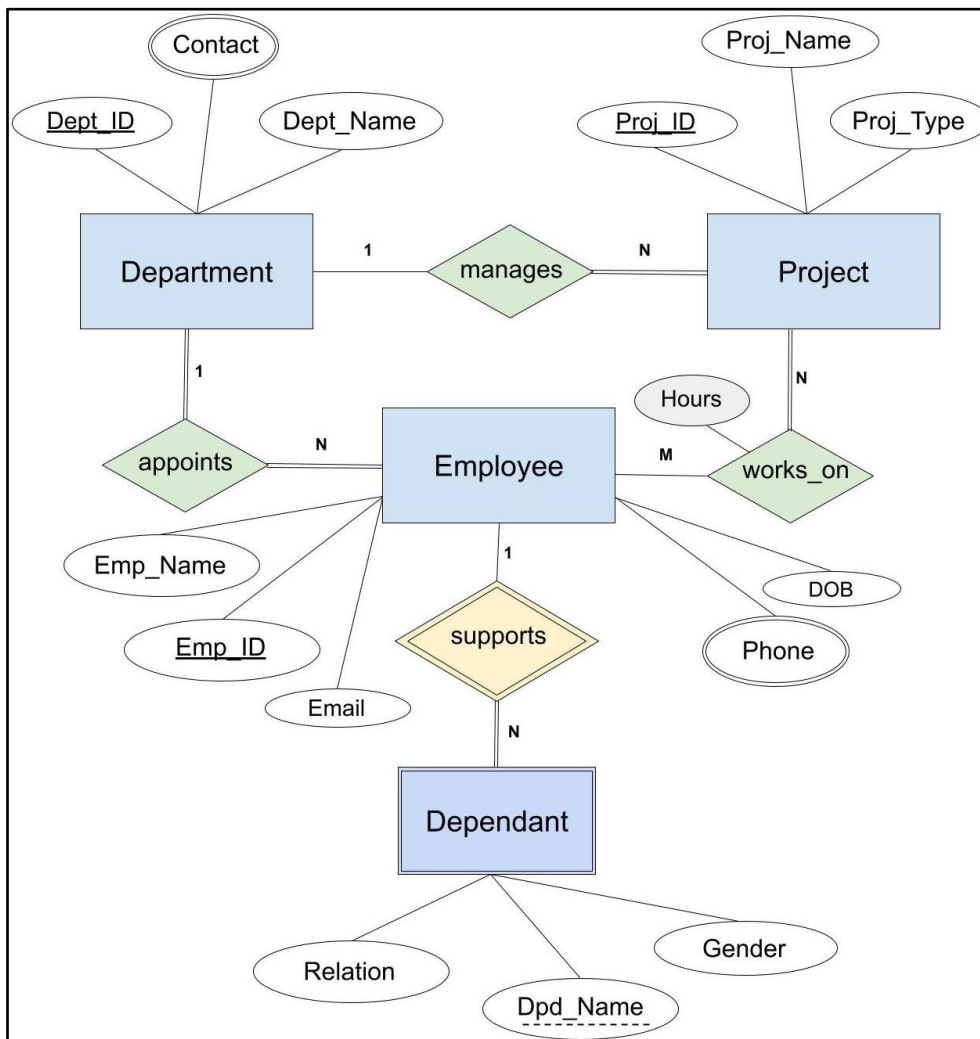
5. Draw and explain conceptual data model with E-R diagram for employee management system.

Ans.

Conceptual Data Model – Employee Management System

A **conceptual data model** represents the main entities, their attributes, and relationships in a system. For an Employee Management System, the main entities can be **Employee, Department, Project, and Manager.**

E-R Diagram



Relationships

1. Department – Employee:

One department can have **many employees**, while each employee belongs to one department.

Cardinality: 1 : N

2. Employee – Project:

An employee can work on **many projects**, and a project can have **many employees**.

Cardinality: M : N

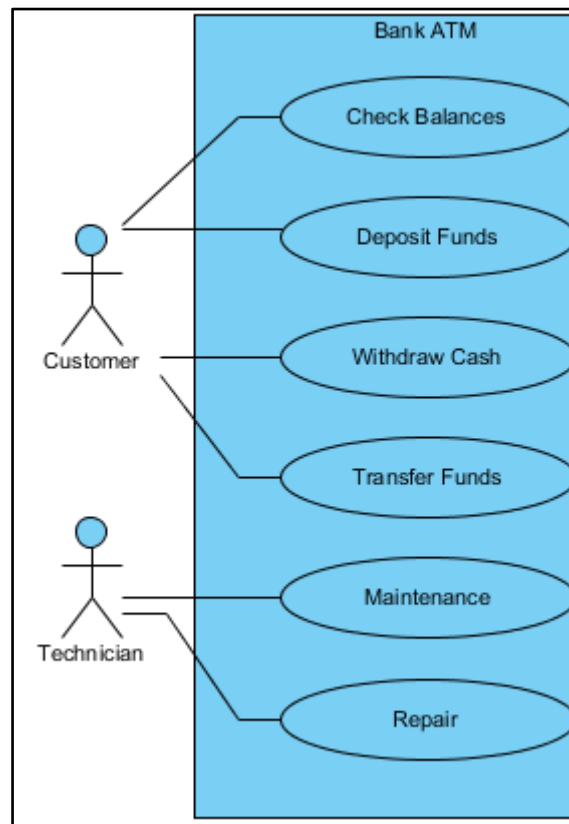
Explanation

- **Employee** is the main entity containing employee details.
- **Department** stores department information.
- **Project** stores project details.
- **PK (Primary Key)** uniquely identifies each entity.
- **FK (Foreign Key)** connects related entities.

Thus, the conceptual E-R model provides a **high-level view of the data and relationships** in an Employee Management System.

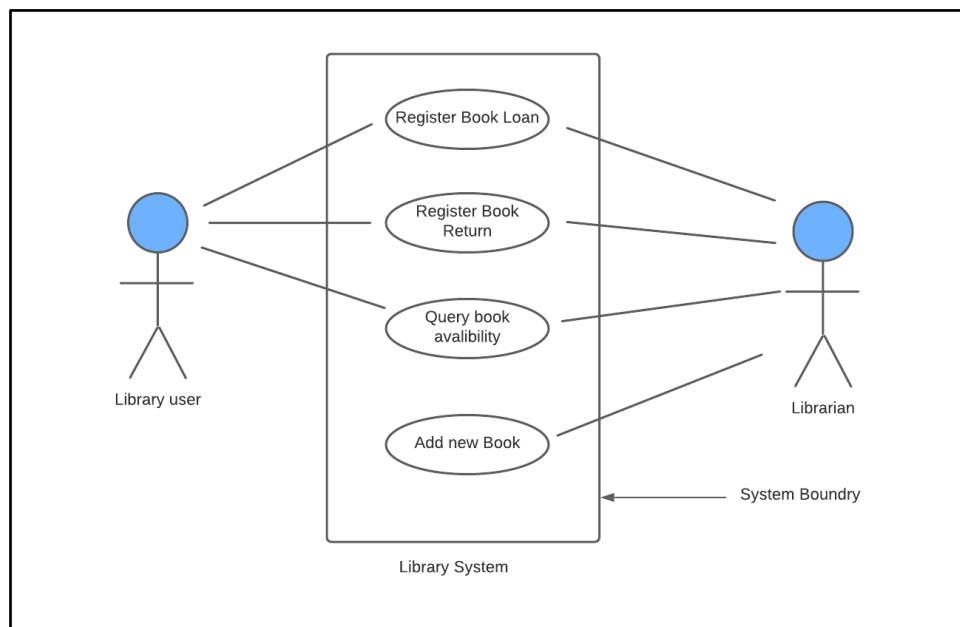
6. Draw use-case diagram for ATM system with minimum four use cases and two actors.

Ans.



7. Sketch use-case diagram for Library management with minimum four use cases and two actors.

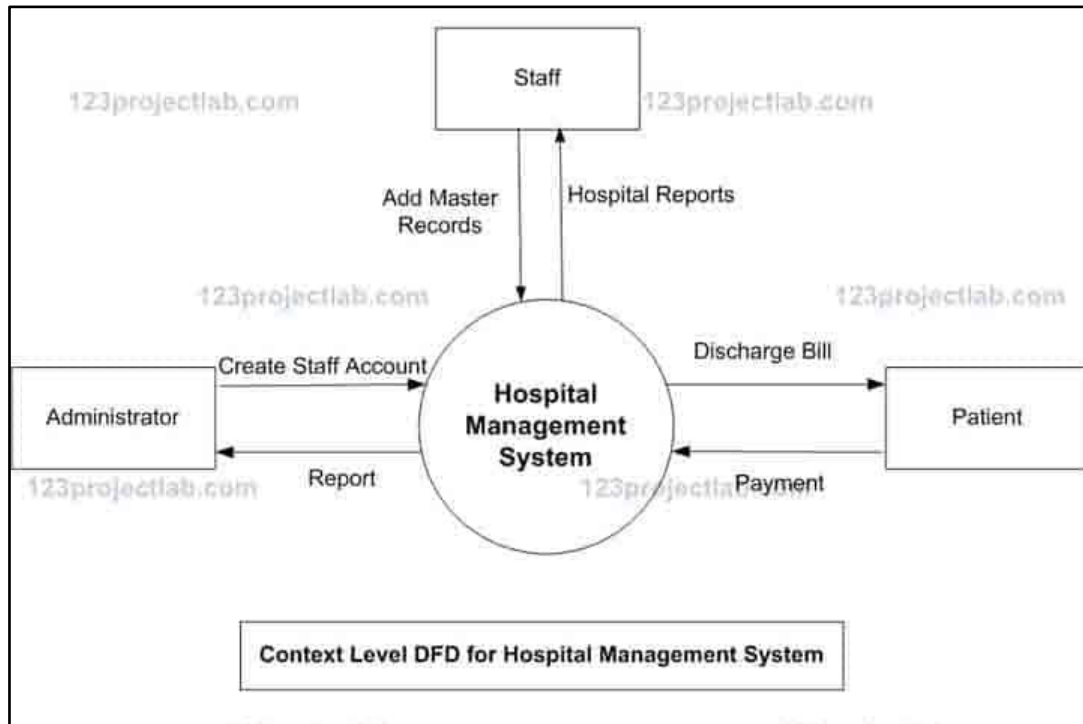
Ans.



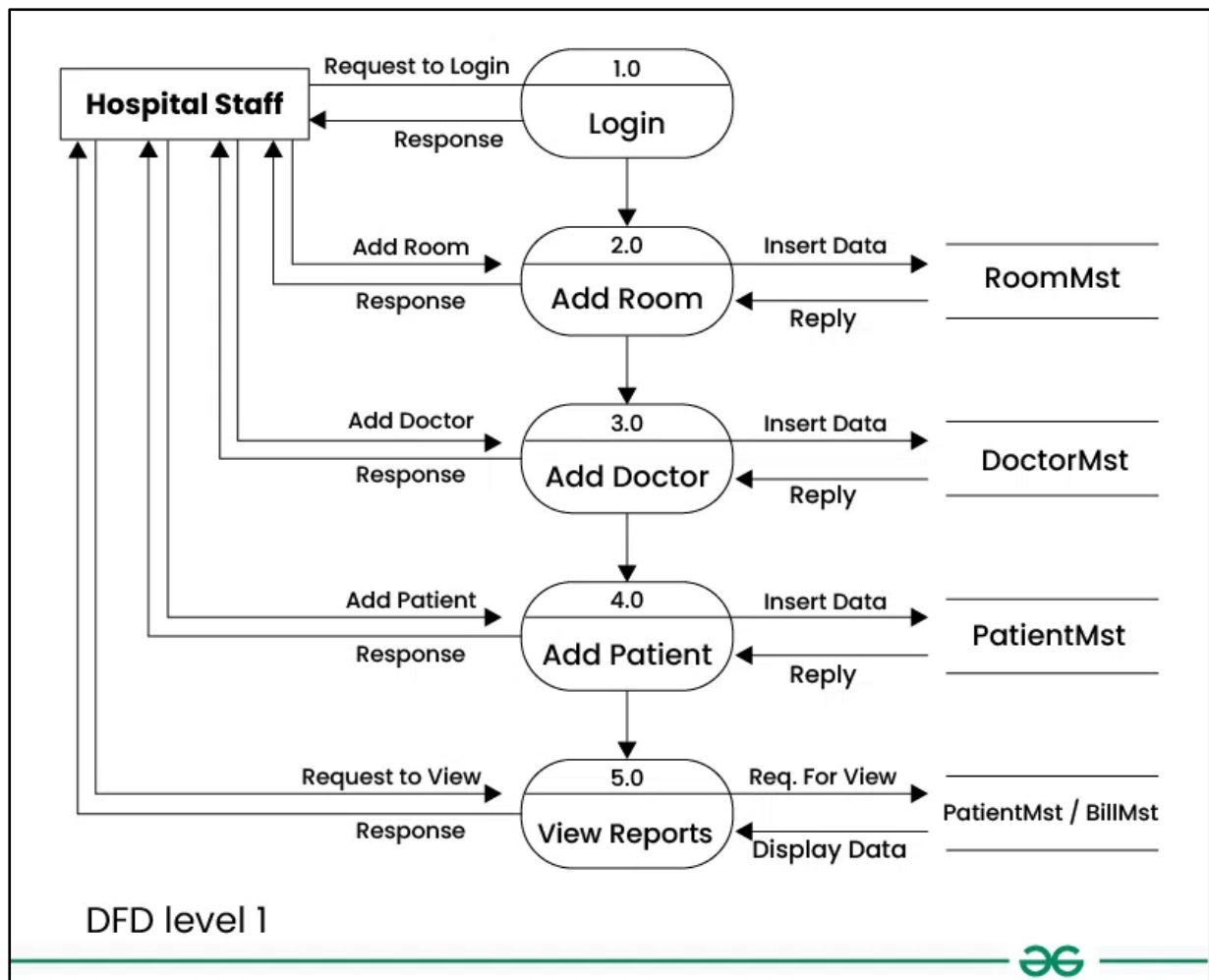
8. Draw Level 0 and Level 1 DFD for : Hospital Management System, Library Management System, Online Shopping System, Hotel Management System, Book Donation Portal.

Ans.

Hospital Management System

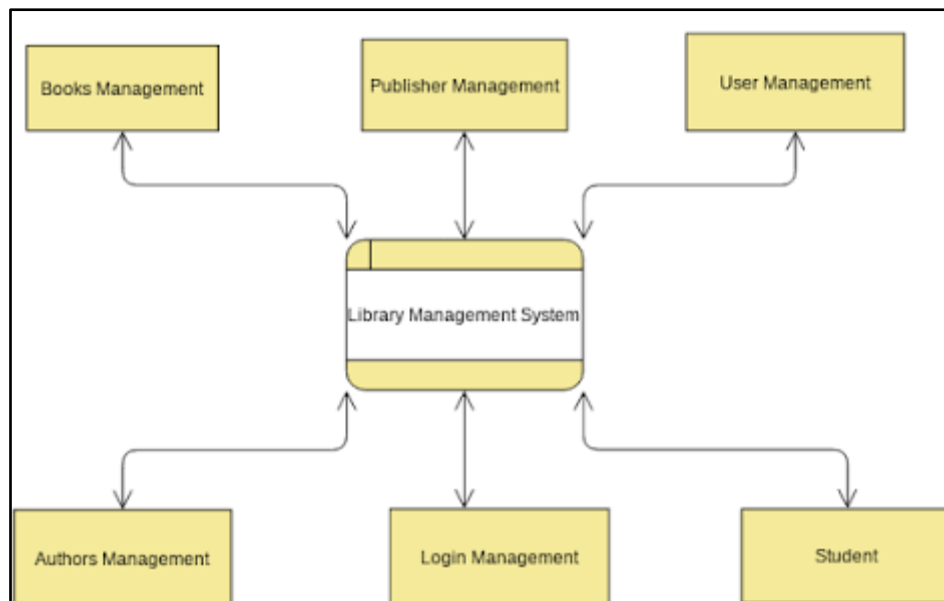


Level 0 DFD

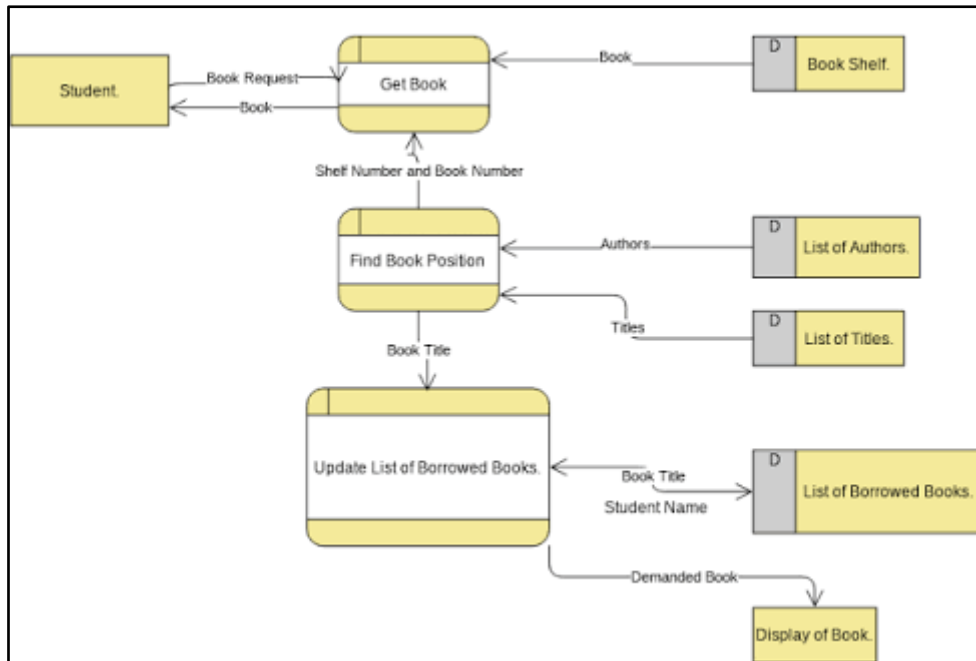


Level 1 DFD

Library Management System

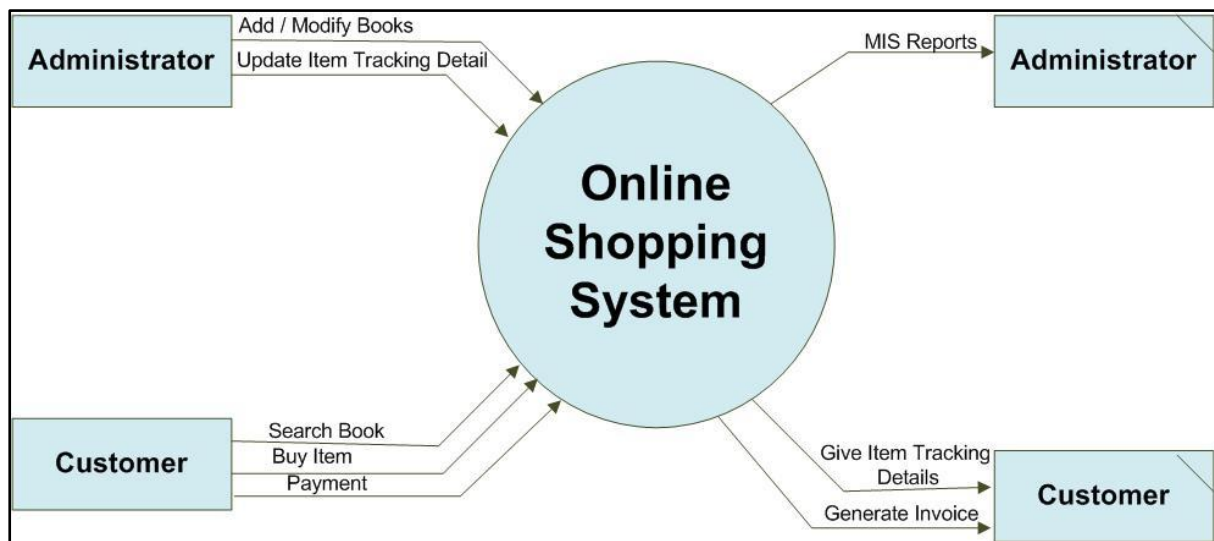


Level 0 DFD

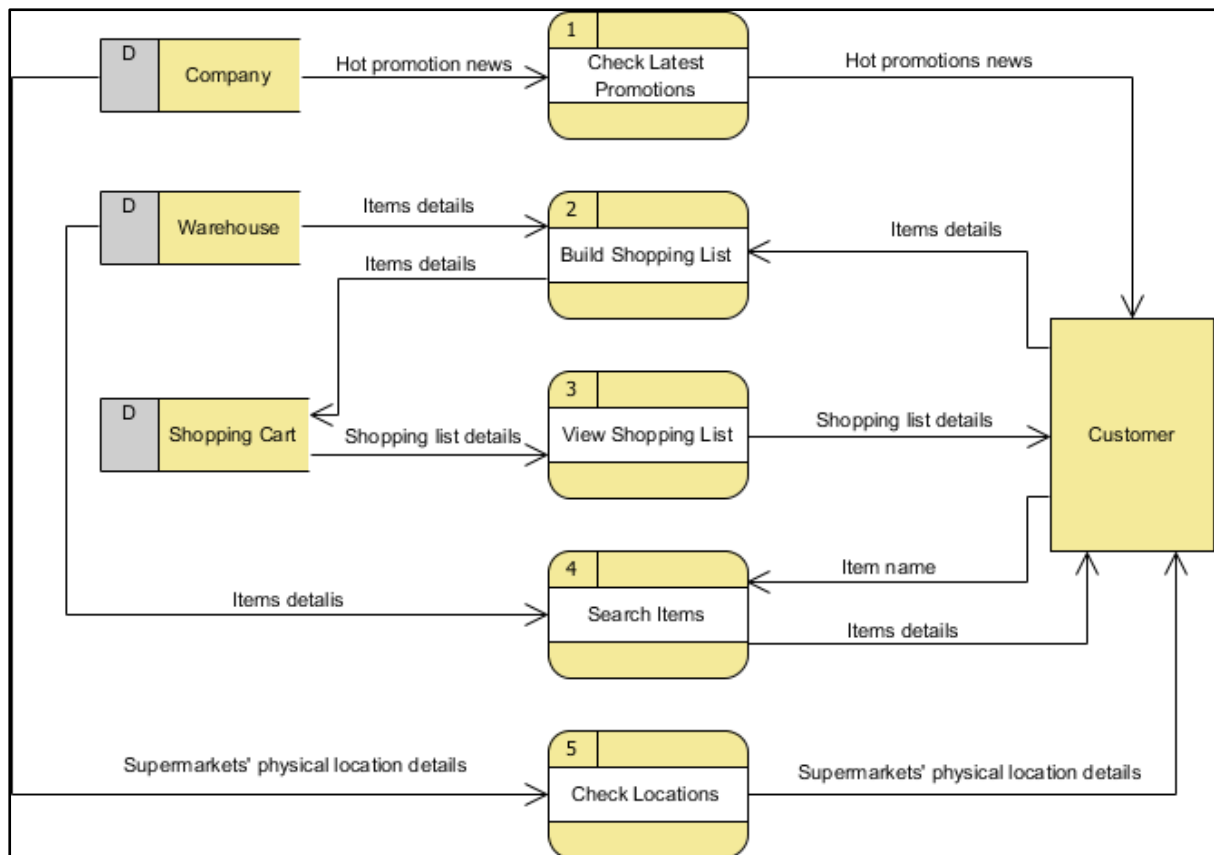


Level 1 DFD

Online Shopping System

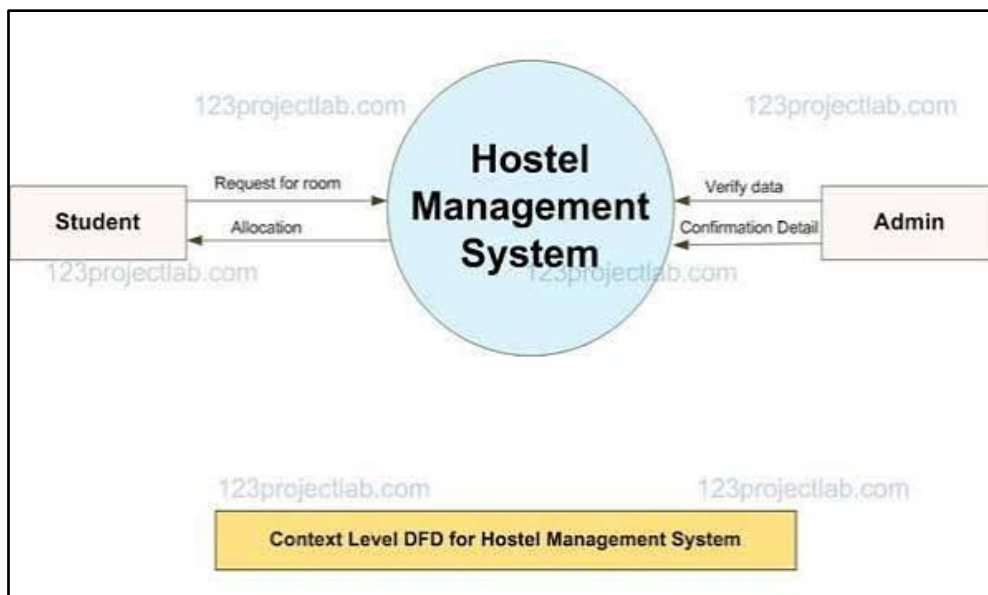


Level 0 DFD

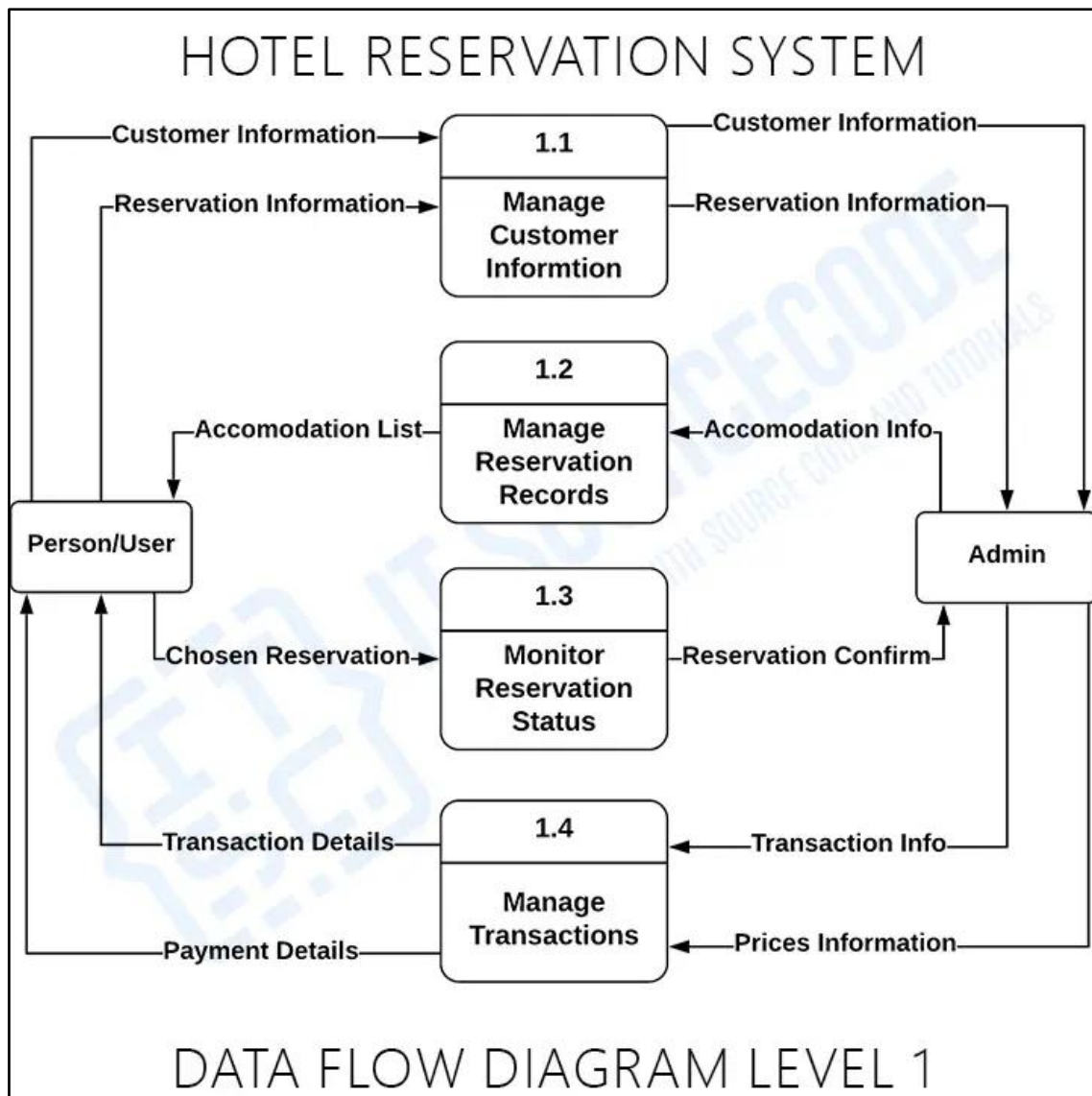


Level 1 DFD

Hotel Management System

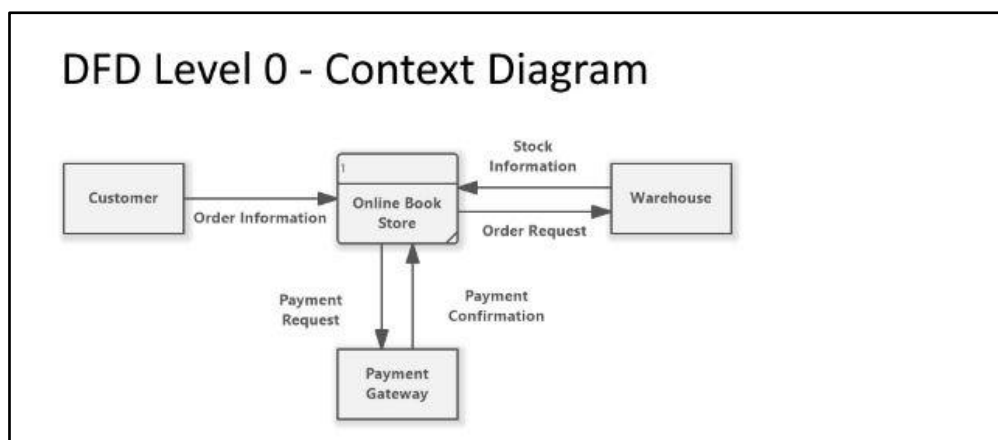


Level 0 DFD

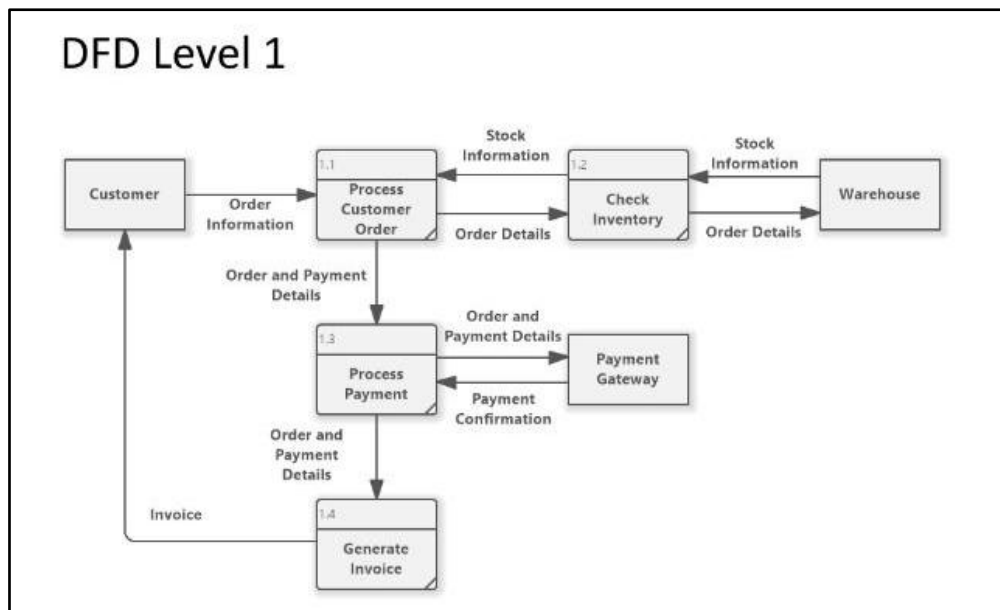


Level 1 DFD

Book Donation Portal



Level 0 DFD



Level 1 DFD

DFD Symbols

Symbol	Meaning
Rectangle	External entity (user, admin, doctor)
Circle	Process that transforms data
Cylinder	Data store or database
Arrow	Direction of data flow

Unit 4

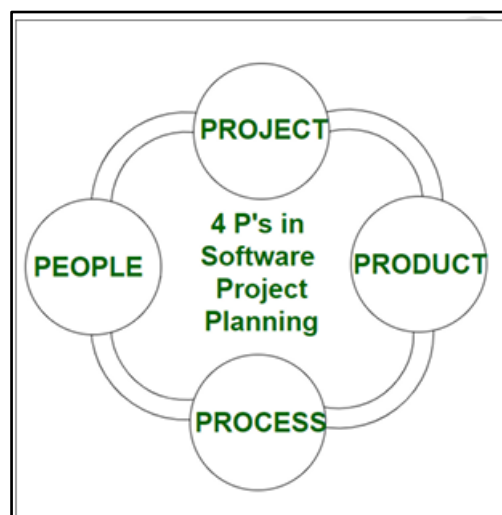
1. State different types of estimations in Software Projects.

Ans.

- **Analogous Estimation** : Compares the current project with similar projects that have been developed in the past to estimate its size.
- **Expert Judgment** : Leverages the experience of experts to provide estimates based on their knowledge of similar systems and functionalities.
- **Work Breakdown Structure (WBS)** : Divides the software project into smaller, manageable sub-systems and modules, allowing for size estimation at a more detailed level.
- **Parametric Models** : Uses mathematical models based on historical data and project parameters to predict size. Factors Influencing Size Estimation
- **Project Complexity** : More complex projects require more effort and are thus larger in scope.
- **Requirements and Functionality** : The number and complexity of features defined in the Software Requirements Specification (SRS) document significantly impact the project size.
- **Team Experience and Technology** : The skills of the development team and the chosen development technologies can affect the final size and the efficiency of the estimation process.

2. Describe 4 P's of management spectrum.

Ans.



1. People : The most important component of a product and its successful implementation is human resources. In building a proper product, a well-managed team with clear-cut roles defined for each person/team will lead to the success of the product. We need to have a good team in order to save our time, cost, and effort. Some

assigned roles in software project planning are project manager, team leaders, stakeholders, analysts, and other IT professionals. Managing people successfully is a tricky process which a good project manager can do.

2. Product : As the name inferred, this is the deliverable or the result of the project. The project manager should clearly define the product scope to ensure a successful result, control the team members, as well technical hurdles that he or she may encounter during the building of a product. The product can consist of both tangible or intangible such as shifting the company to a new place or getting a new software in a company.

3. Process : In every planning, a clearly defined process is the key to the success of any product. It regulates how the team will go about its development in the respective time period. The Process has several steps involved like, documentation phase, implementation phase, deployment phase, and interaction phase.

4. Project : The last and final P in software project planning is Project. It can also be considered as a blueprint of process. In this phase, the project manager plays a critical role. They are responsible to guide the team members to achieve the project's target and objectives, helping & assisting them with issues, checking on cost and budget, and making sure that the project stays on track with the given deadlines.

3. Describe function point metrics for size estimation.

Ans.

- In software engineering, a function point (FP) is a unit of measurement that quantifies the business functionality delivered by a software application, used to estimate project size, effort, cost, and schedule.
- FPA involves counting five types of functions—external inputs, external outputs, external inquiries, internal logical files, and external interface files—assigning them complexity weights (simple, average, complex), and summing these to get an unadjusted function point total.
- This is then adjusted by 14 general system characteristics to get the final FP count, providing a standardized and user-focused metric for project planning and analysis.

4. Describe RMMM strategy with example.

Ans.

- To apply the RMMM (Risk Mitigation, Monitoring, and Management) strategy in software development, first identify potential risks, then assess their probability and impact, mitigate them with strategies like avoidance, reduction, or acceptance, and finally monitor those risks and their mitigation efforts continuously while having management and contingency plans in place to respond to unforeseen events.

- A risk management technique is usually seen in the software Project plan. This can be divided into Risk Mitigation, Monitoring, and Management Plan (RMMM). In this plan, all works are done as part of risk analysis. As part of the overall project plan project manager generally uses this RMMM plan.

Example :

Let us understand RMMM with the help of an example of high staff turnover.

1. Risk Mitigation : To mitigate this risk, project management must develop a strategy for reducing turnover. The possible steps to be taken are :

- Meet the current staff to determine causes for turnover (e.g., poor working conditions, low pay, competitive job market).
- Mitigate those causes that are under our control before the project starts.
- Once the project commences, assume turnover will occur and develop techniques to ensure continuity when people leave.
- Organize project teams so that information about each development activity is widely dispersed.
- Define documentation standards and establish mechanisms to ensure that documents are developed in a timely manner.
- Assign a backup staff member for every critical technologist.

2. Risk Monitoring : As the project proceeds, risk monitoring activities commence. The project manager monitors factors that may provide an indication of whether the risk is becoming more or less likely. In the case of high staff turnover, the following factors can be monitored :

- General attitude of team members based on project pressures.
- Interpersonal relationships among team members.
- Potential problems with compensation and benefits.
- The availability of jobs within the company and outside it.

3. Risk Management : Risk management and contingency planning assumes that mitigation efforts have failed and that the risk has become a reality. Continuing the example, the project is well underway, and a number of people announce that they will be leaving. If the mitigation strategy has been followed, backup is available, information is documented, and knowledge has been dispersed across the team. In addition, the project manager may temporarily refocus resources (and readjust the project schedule) to those functions that are fully staffed, enabling newcomers who must be added to the team to "get up to the speed".

5. Use COCOMO model to calculate :

i) Efforts ii) Development Time iii) Average staff size iv) Productivity.

If estimation size of product is 400 KLOC using organic model.

Ans.

Given :

- Size of product = 400 KLOC
- COCOMO model = Organic
- Organic mode constants:
 - $a = 2.4$
 - $b = 1.05$
 - $c = 2.5$
 - $d = 0.38$

COCOMO Formulas

1. Effort (E):

$$E = a(KLOC)^b$$

2. Development Time (T):

$$T = c(E)^d$$

3. Average Staff Size:

$$Staff = \frac{E}{T}$$

4. Productivity:

$$Productivity = \frac{KLOC}{E}$$

Calculation : -

i) Effort:

$$E = 2.4(400)^{1.05}$$
$$E \approx 1377.8 \text{ Person-Months}$$

ii) Development Time:

$$T = 2.5(1377.8)^{0.38}$$
$$T \approx 41.7 \text{ months}$$

iii) Average Staff Size:

$$Staff = \frac{1377.8}{41.7}$$

$Staff \approx 33.0 \text{ persons}$

iv) Productivity:

$$Productivity = \frac{400}{1377.8}$$

$Productivity \approx 0.29 \text{ KLOC/Person-Month}$

i. Efforts taken are approximately 1377.8 Person-Months.

ii. Development time approximately 41.7 Months.

iii. Average staff size approximately 33.0 Persons.

iv. Productivity approximately 0.29 KLOC/Person-Month

6. Specify following cost directives of COCOMO :

i) Product Attributes

ii) Hardware Attributes

Ans.

Product attributes :

- Required Software Reliability extent
- Size of the application database
- The complexity of the product

Hardware attributes :

- Run-time performance constraints
- Memory constraints
- The volatility of the virtual machine environment
- Required turnabout time

7. Describe following project cost estimation approaches :

i) Heuristic

ii) Empirical

Ans.

i) Heuristic Approach

- It is based on **rules of thumb, expert judgment, and past experience**.
- Experienced project managers estimate cost using similar previous projects.
- It is useful when **accurate historical data is not available**.
- It is simple and quick, but can be **subjective and less accurate**.

Example: An experienced manager estimates that a similar project will cost approximately ₹5 lakh based on previous projects.

ii) Empirical Approach

- It uses **mathematical models and historical project data** to estimate cost.
- Cost is calculated using relationships between factors such as **size, effort, time, and cost**.
- Models like **COCOMO** are examples of empirical estimation.
- It is more **systematic and objective** than the heuristic approach.

Example: Using COCOMO, the cost is estimated from the project's size in **KLOC** and development characteristics.

8. Explain line of code metrics for size estimation.

Ans.

- Lines of Code (LOC) is a software metric used to measure the size of a computer program by counting the number of lines in the text of the program's source code. It is one of the oldest and most basic metrics in software engineering.

- Measures the total number of lines in the source code, which can be quantified as KLOC (thousands of lines of code).

9. Describe a cocomo and cocomo-II models.

Ans.

COCOMO Model

COCOMO (Constructive Cost Model) is a software cost estimation model developed by **Barry Boehm**. It estimates **effort, development time, and staffing** based mainly on the size of the software in **KLOC (Kilo Lines of Code)**.

Basic COCOMO Formula:

- **Effort** = $a \times (\text{KLOC})^b$ Person-Months
- **Development Time** = $c \times (\text{Effort})^d$ Months

COCOMO Project Modes:

1. **Organic** – Small and simple projects with experienced teams.
2. **Semi-detached** – Medium-size projects with mixed experience.
3. **Embedded** – Complex projects with strict hardware/software constraints.

COCOMO II Model

COCOMO II is an improved version of COCOMO designed for **modern software development**. It supports iterative development, reuse, object-oriented development, and modern programming practices.

Main Features:

- Uses **Size, Scale Factors, and Cost Drivers**.
- Provides more accurate estimation for modern projects.
- Supports **reuse of existing software components**.
- Can be applied at different stages of software development.

Unit 5

1. Define PERT and CPM.

Ans.

PERT:

The Programme Evaluation and Review Technique (PERT) is a project management tool that uses a flowchart or network diagram to break down a project into tasks, estimate task durations, and identify dependencies between tasks.

CPM:

A technique that identifies the longest sequence of dependent tasks in a project, determining the minimum time needed for completion and highlighting critical activities.

2. Define Project Scheduling.

Ans.

Project Scheduling:

Project scheduling in software engineering is the systematic process of organizing and sequencing project tasks, defining dependencies, estimating durations, allocating resources, and assigning timelines to achieve project goals within a specified timeframe.

3. What is Gantt Chart? State its Advantages.

Ans.

Visual tools that display project tasks and durations on a timeline, providing a clear overview of the project schedule. Following are the advantage of Gantt Chart.

1. Easy identification of tasks: The Gantt Chart helps us to easily identify the various tasks, their inter-relation, and the Progress of the tasks.

2. Helps to evaluate critical resources utilization: We can evaluate the critical Resources and their utilization, Dependencies, and Impacts of schedule changes in the Gantt Chart. Along with this, we can also create Visual alerts in case of scheduling conflicts.

3. Helps to manage tasks order: The Gantt chart equips the project experts with the capabilities to manage the order in which tasks need to be carried out so that we can easily identify the priority of the activities.

4. Explain Work breakdown Structure with suitable diagram. (4M)

Ans.

Work breakdown structure:

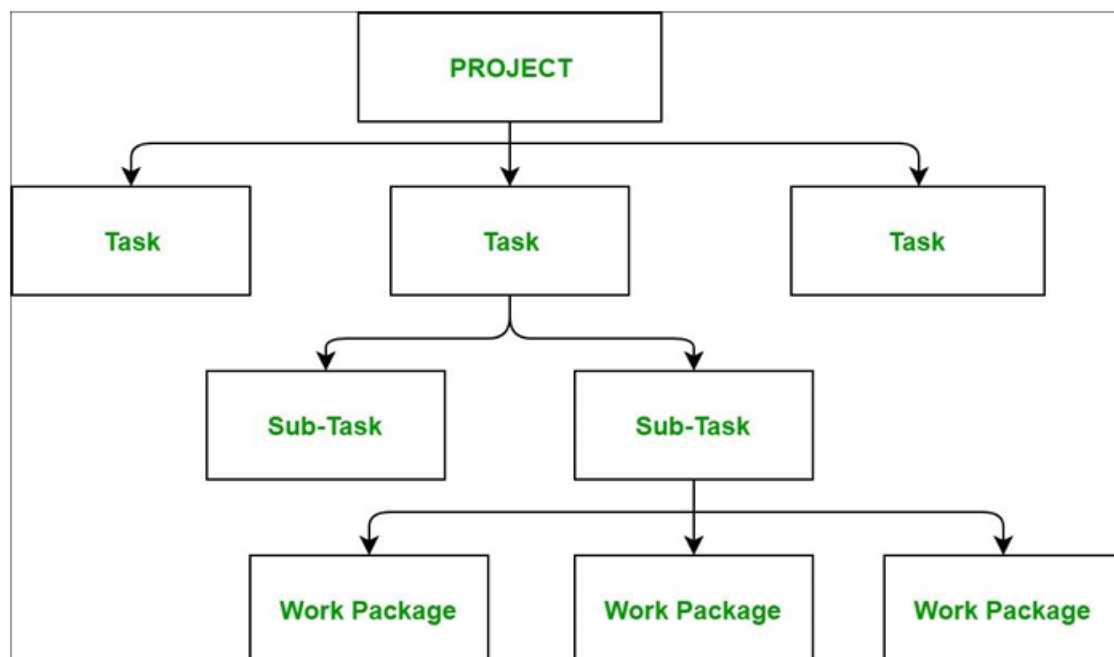
A Work Breakdown Structure includes dividing a large and complex project into simpler, manageable, and independent tasks. The root of this tree (structure) is labeled by the Project name itself. For constructing a work breakdown structure, each node is recursively decomposed into smaller sub activities, until at the leaf level, the activities become undividable and independent. It follows a Top Down approach.

Steps Work Breakdown Structure:

Step 1: Identify the major activities of the project.

Step 2: Identify the sub-activities of the major activities.

Step 3: Repeat till undividable, simple, and independent activities are created.



Construction of Work Breakdown Structure

1. Firstly, the project managers and top level management identifies the main deliverables of the project.
2. After this important step, these main deliverables are broke down into smaller higher-level tasks and this complete process is done recursively to produce much smaller independent tasks.
3. It depends on the project manager and team that upto which level of detail they want to break down their project.
4. Generally the lowest level tasks are the most simplest and independent tasks and takes less than two weeks worth of work.
5. Hence, there is no rule for upto which level we may build the work breakdown structure of the project as it totally depends upon the type of project we are working on and the management of the company.

6. The efficiency and success of the whole project majorly depends on the quality of the Work Breakdown Structure of the project and hence, it implies its importance.

5. State and Explain any 4 Principles of project scheduling.

Ans.

Principles of Project Scheduling

1. **Compartmentalization:**
The project is divided into smaller, manageable tasks or activities. This makes planning and monitoring easier.
2. **Interdependency:**
The dependencies between different activities are identified. This helps to decide the correct sequence in which tasks should be performed.
3. **Time Allocation:**
A specific start date, end date, and duration are assigned to each activity to ensure timely completion of the project.
4. **Effort Validation:**
The estimated effort and resources required for each activity are checked to ensure that the schedule is realistic and achievable.
5. **Defined Responsibilities:**
Assign each task to a specific team member or person so that everyone knows their responsibilities.
6. **Defined Outcomes:**
Clearly define the expected result or deliverable of each activity so that progress can be measured easily.

6. Prepare timeline chart for ATM machine.

Ans.

A timeline chart for an ATM machine's operation isn't a historical timeline of its invention, but rather a sequence of events in a single transaction, starting with the user inserting a card, authenticating with a PIN, selecting a transaction, entering an amount, and finally receiving cash and a receipt. The ATM communicates with the bank's server at multiple points to validate the card and account balance, ensuring the transaction's security and successful completion.

Here is a step-by-step timeline for a typical ATM cash withdrawal:

1. **Insert Card:** The user inserts their debit or credit card into the ATM.
2. **Card Validation:** The ATM reads the card and sends the information to the bank's server to verify its validity.
3. **PIN Entry:** The bank server confirms the card is valid and sends a message to the ATM. The ATM then prompts the user to enter their PIN.
4. **PIN Validation:** The entered PIN is sent to the bank server for authentication.

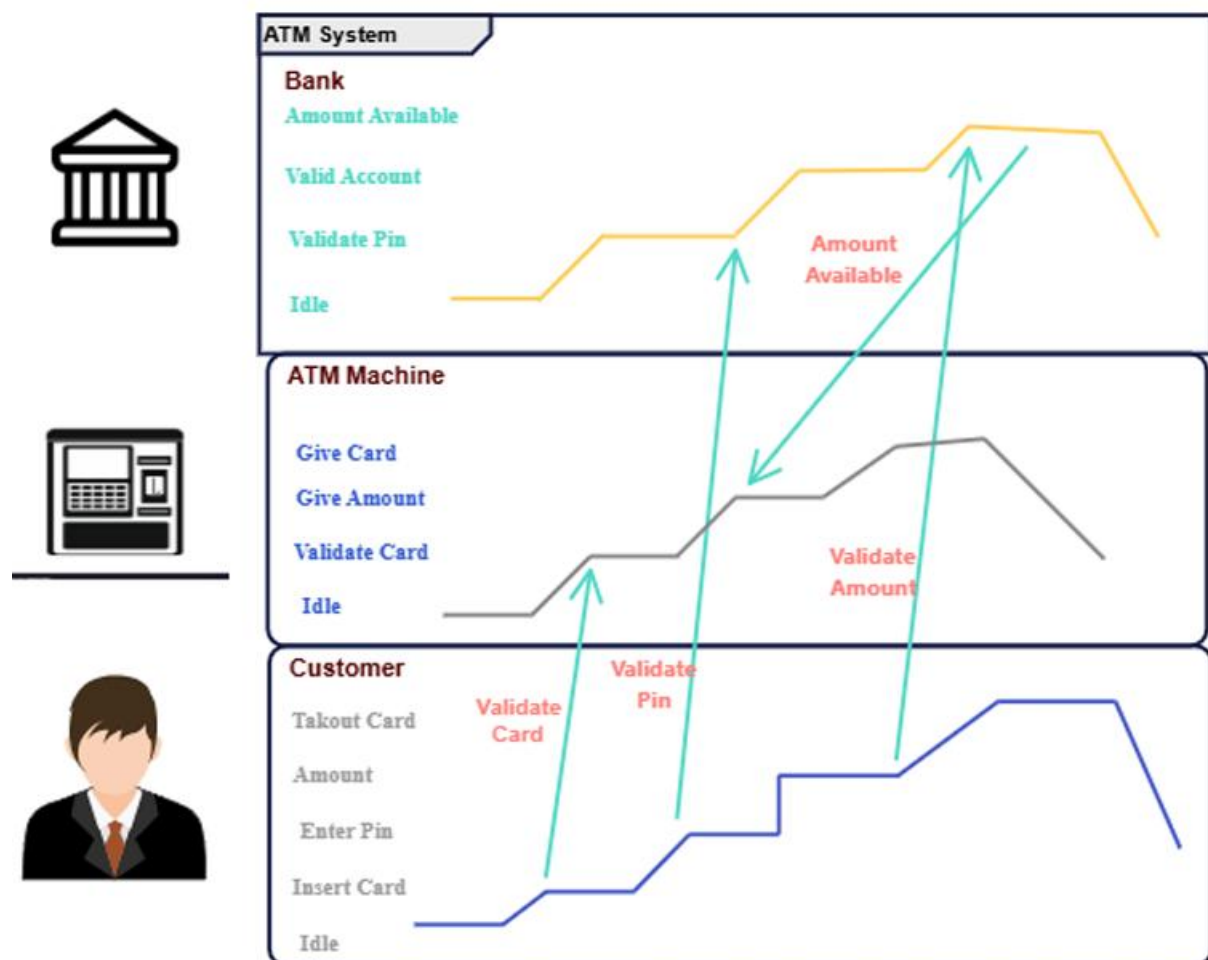
5. Transaction Selection: If the PIN is valid, the bank server sends a confirmation to the ATM, which then displays the main menu. The user selects their desired transaction type (e.g., withdrawal) and account type.

6. Withdrawal Amount: The user enters the desired withdrawal amount.

7. Fund Check: The ATM sends the withdrawal request to the bank server to check for sufficient funds in the account.

8. Dispense Cash: If funds are available, the bank server approves the transaction, the ATM counts the cash, and then dispenses it to the user.

9. Receipt & End Transaction: The ATM prompts the user if they want a receipt. After the user's choice, the ATM ejects the card, and the transaction is complete.



7. Prepare macro time line chart for 15 days of college management system (5 days a week).

Ans.

A macro timeline for a 15-day, 5-days-a-week college management system project, when considering SDLC phases, would allocate roughly 1-2 days per week to Requirements Gathering, Analysis, and Design, 1-2 days per week to Development and Testing, and 1 day per week to Deployment and Maintenance, with the remaining days

for initial planning and finalization. This high-level view helps in understanding the broad project flow over the 15-day period.

Macro Time Line (15 Days / 3 Weeks)

This timeline assumes a 5-day work week (Monday to Friday).

- **Week 1: Foundation & Planning**

- **Day 1-2 (Monday-Tuesday):** Project Initiation & Requirements Gathering. Kick-off meeting, defining the project scope and high-level requirements for the college management system.

- **Day 3-4 (Wednesday-Thursday):** Requirements Analysis & High-Level Design. Analyzing the gathered requirements and sketching a basic, high-level architecture for the system.

- **Day 5 (Friday):** Detailed Design & Planning. Begin creating detailed design specifications for key modules and finalize a detailed project plan for the subsequent weeks.

- **Week 2: Development & Initial Testing**

- **Day 6-7 (Monday-Tuesday):** Module Development (Phase 1). Developers start building critical modules, such as student registration and course management.

- **Day 8-9 (Wednesday-Thursday):** Module Development (Phase 2) & Unit Testing. Continue developing other modules, like faculty profiles and attendance tracking, and begin unit testing completed modules.

- **Day 10 (Friday):** Integration Testing (Initial). Integrate the developed modules and conduct initial integration testing to find and fix early integration issues.

- **Week 3: Testing, Deployment & Wrap-up**

- **Day 11-12 (Monday-Tuesday):** System Testing & User Acceptance Testing (UAT) Preparation. Conduct comprehensive system testing for the integrated system and prepare for user acceptance testing.

- **Day 13 (Wednesday):** User Acceptance Testing (UAT). Key stakeholders or end-users test the system to ensure it meets their needs and requirements.

- **Day 14 (Thursday):** Deployment & Training (Initial). Deploy the approved system and conduct initial training for administrators and key users.

- **Day 15 (Friday):** Final Review & Handover. Conduct a final project review, address any immediate post-deployment issues, and formally hand over the system.

Unit 6

1. Differentiate between Software Quality Management and Software Quality Assurance.

Ans.

Feature	Software Quality Management (SQM)	Software Quality Assurance (SQA)
Scope	Broad; the entire quality system for the organization or project.	Narrower; a specific process within the SQM framework.
Focus	Strategic planning, setting standards, and overall quality goals.	Process adherence, preventing defects, and verifying standards are followed.
Nature	The "overall system" and strategic direction for quality.	A "technique" or system to assure quality through planned activities.
Goal	Achieve high-quality standards across the entire organization.	Ensure processes are executed correctly to prevent defects and build quality in.
Timeframe	Covers all stages of project and product delivery.	Operates throughout the entire Software Development Life Cycle (SDLC).

2. Apply the phases of Software Quality Assurance in software development project.

Ans.

Software Quality Assurance (SQA) encompasses planning the overall quality strategy, actively performing SQA activities like reviews and testing, performing audits to verify process adherence, and conducting reviews to evaluate results and ensure alignment with standards. The process is iterative and continuous, typically following a **Plan-Do-Check-Act** cycle to ensure software quality throughout the development lifecycle and maintain customer satisfaction.

1. Planning

- **Define Objectives:** Establish clear goals and objectives for software quality, aligning them with business goals and customer expectations.
- **Create the SQA Plan:** Develop a comprehensive plan that outlines the processes, metrics, and resources needed to achieve the defined quality goals.
- **Set Success Metrics:** Determine key performance indicators (KPIs) such as defect density, test coverage, and maintenance costs to measure quality.
- **Establish Standards:** Identify and adopt relevant industry standards, processes, and regulatory requirements to ensure compliance.

2. Activities (Do)

- **Requirement Gathering:** Participate in the gathering and understanding of both functional and non-functional software requirements.

- **Process Support:** Support the development team by providing input into the software process description and adhering to defined processes.
- **Test Strategy & Execution:** Develop and implement a multi-testing strategy, including functional, regression, integration, and performance tests, and execute them to find defects.
- **Defect Tracking:** Track and report any defects found during testing to the senior management for resolution.

3. Audits (Check)

- **Verify Process Adherence:** Conduct audits to verify that the development team is following the defined software processes and standards.
- **Work Product Inspection:** Audit software work products, such as design documents, to ensure they comply with the defined standards and initial assumptions.
- **Document Compliance:** Record any non-compliance identified during the audit and report it to the management for corrective action.

4. Reviews (Act)

- **Formal Technical Reviews:** Conduct formal reviews of software components and designs to identify potential issues early in the development cycle.
- **Evaluate Results:** Evaluate the outcomes of the implemented plan, checking if the set goals and objectives have been reached.
- **Continuous Improvement:** Make any necessary changes based on the findings from the check and audit phases to refine the process and achieve better results.

3. Describe six sigma. State operations in DMADV/DMAIC.

Ans.

Six Sigma:

- Six Sigma is a methodology that helps organizations in making their process better and more efficient by identifying and removing errors and variations.
- Variations in processes can lead to errors, these errors can lead to product defects and product defects can lead to poor customer satisfaction.
- By reducing variation and errors Six Sigma can reduce process costs and increase customer satisfaction.
- Six Sigma was introduced in 1986 by an American Engineer Bill Smith. He introduced this term while working at Motorola.
- Industries like manufacturing, service industry, government agencies, aerospace, and e commerce use Six Sigma to improve their processes and product quality.

Methodologies & Frameworks

- **DMAIC (Define, Measure, Analyze, Improve, Control):** Used for improving existing software development processes.

- **DMADV (Define, Measure, Analyze, Design, Verify):**

Ideal for designing new software systems or processes to meet Six Sigma standards from the beginning.

4. Describe CMMI. Give significance of each level

Ans.

CMMI: Levels:

The five CMMI (Capability Maturity Model Integration) levels in software engineering represent an organization's journey toward process excellence, from Level 1 (Initial) where processes are ad hoc, to Level 5 (Optimizing), focused on continuous improvement and innovation. The levels are: Initial, Managed, Defined, Quantitatively Managed, and Optimizing.

Here is a breakdown of each CMMI maturity level:

• Level 0: Incomplete

- Describes processes that are either non-existent or incomplete, leaving work unfinished and unaddressed.

• Level 1: Initial

- Processes are characterized by their unpredictability and often reactive nature, with little to no formal process in place.
- Work is completed, but it often fails to meet project timelines and budgets.

• Level 2: Managed

- Processes are planned, performed, measured, and controlled at the project level, not yet on a company-wide basis.
- This level focuses on establishing basic project management policies and leveraging past project successes.

• Level 3: Defined

- Processes are well-defined, documented, and standardized across the entire organization.
- Organizations at this level proactively understand and document their business goals and can address them through tailored standards and procedures.

• Level 4: Quantitatively Managed

- Processes are measured and controlled using statistical and quantitative techniques.
- This data-driven approach ensures process performance is predictable and aligned with stakeholder needs.

• Level 5: Optimizing

- The organization focuses on continuous improvement through both incremental enhancements and innovative advancements.
- Processes are agile and stable, allowing the organization to adapt to changing environments and pursue new opportunities.